

Centrality Measures

Prof. O-Joun Lee

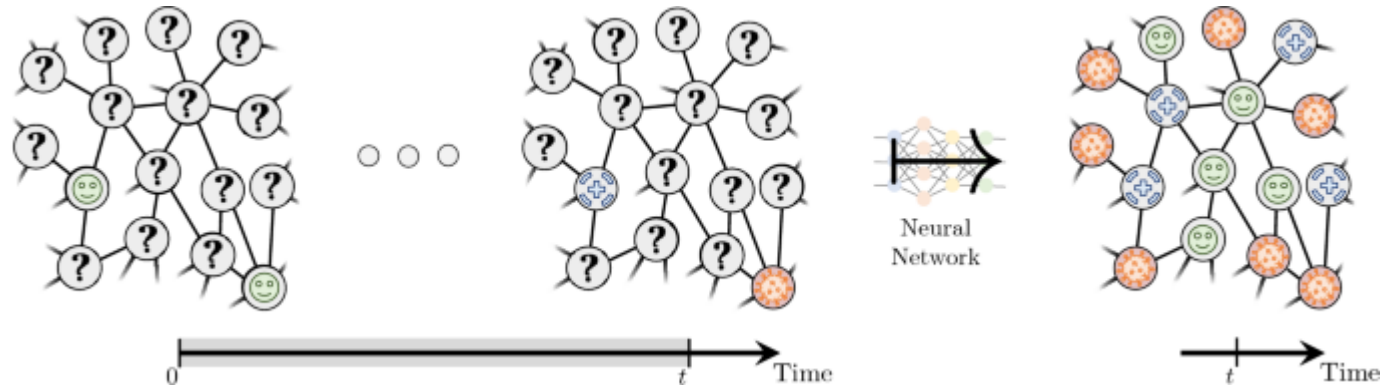
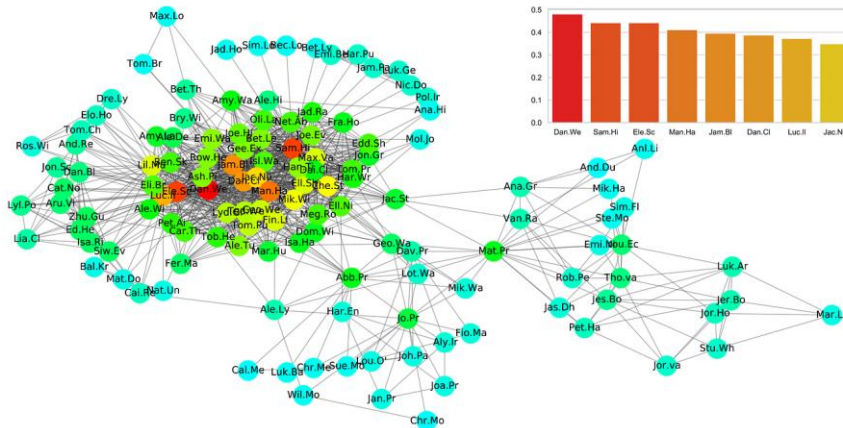
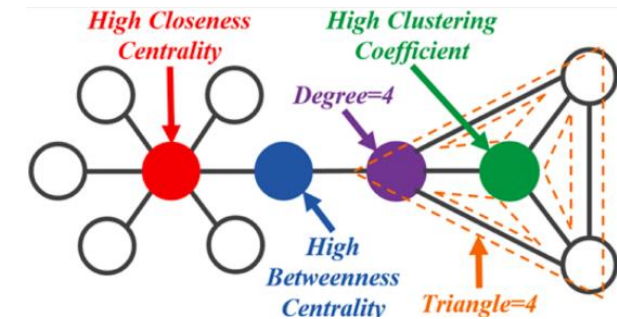
Dept. of Artificial Intelligence,
The Catholic University of Korea
ojlee@catholic.ac.kr

Contents

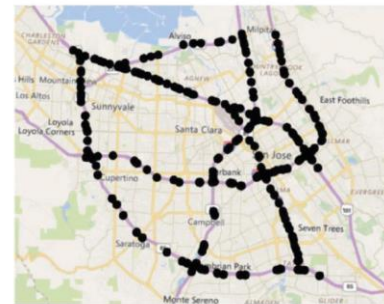
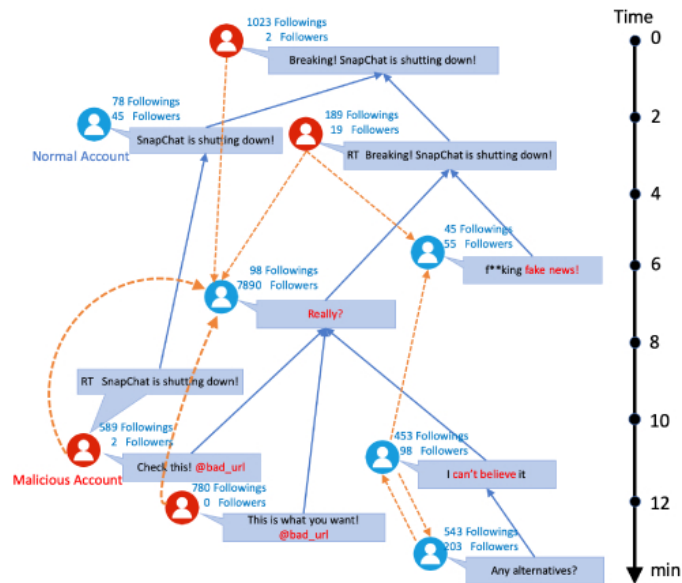
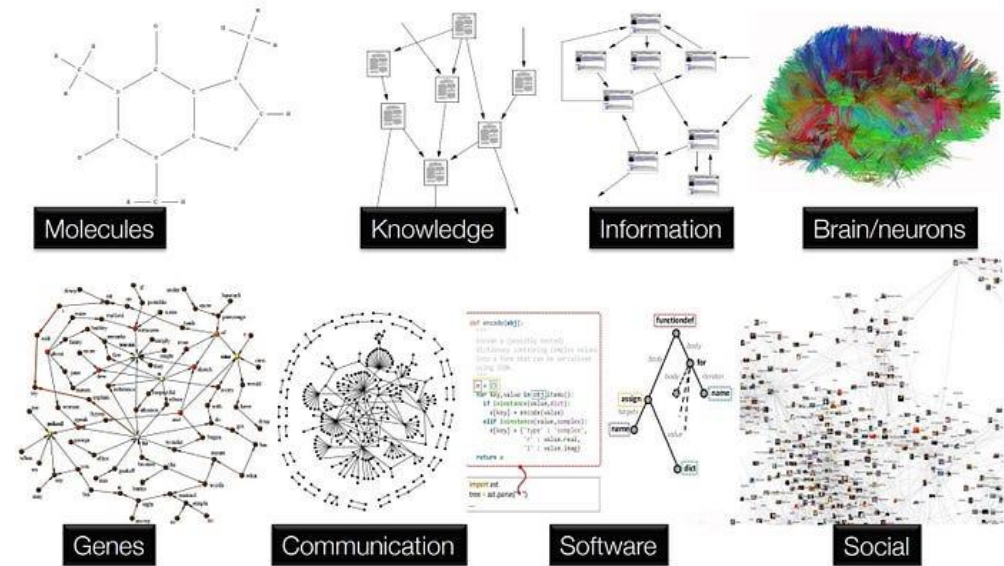


- Centrality Measures
 - Degree Centrality
 - Betweenness Centrality
 - Closeness Centrality
 - Eigenvector Centrality
 - Katz Centrality
 - Participation Coefficient
 - PageRank

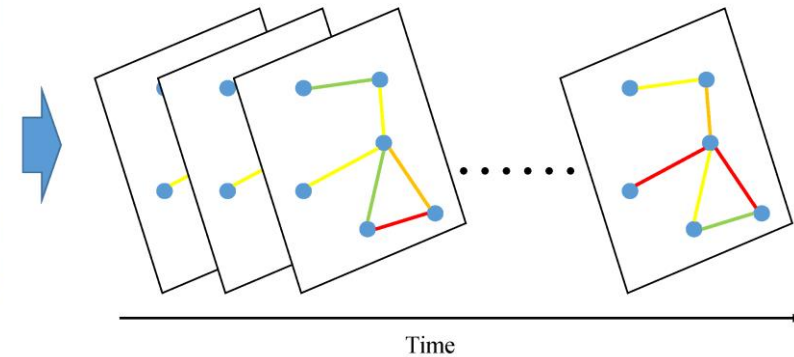
- Knowing the network structure, we can calculate various useful quantities or measures that capture features of network topology
- Centrality measures **represent the most important node** in graphs:
 - The **most influential** person in a social network
 - The **most critical** nodes in an infrastructure
 - The **highest spreaders** of disease
 - How **importance** a web page is



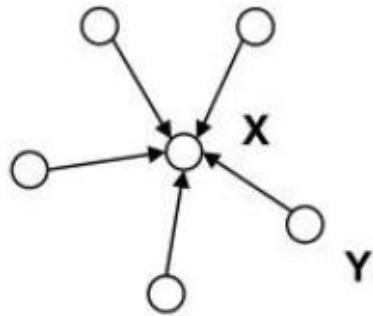
- Another applications of Centrality:
 - Detecting fake news spreaders
 - Protein interaction networks
 - Transportation networks
 - Cybersecurity attack paths
 - Graph Neural Networks (GNNs)



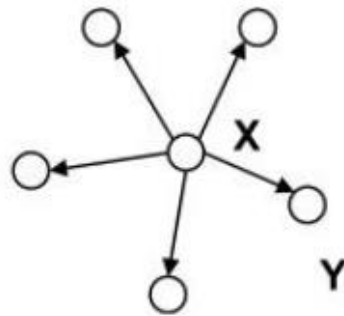
Road Network with Traffic Sensors



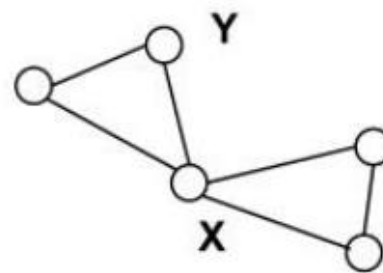
- Who is important based on the network position?
- For example, in each network, 'X' has higher centrality than 'Y' according to a particular measure.



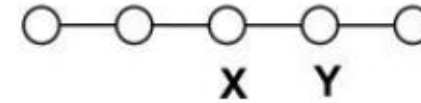
indegree



outdegree

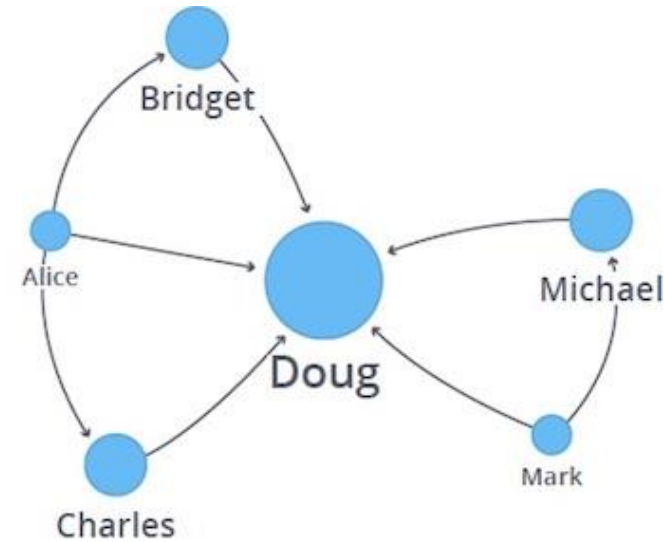
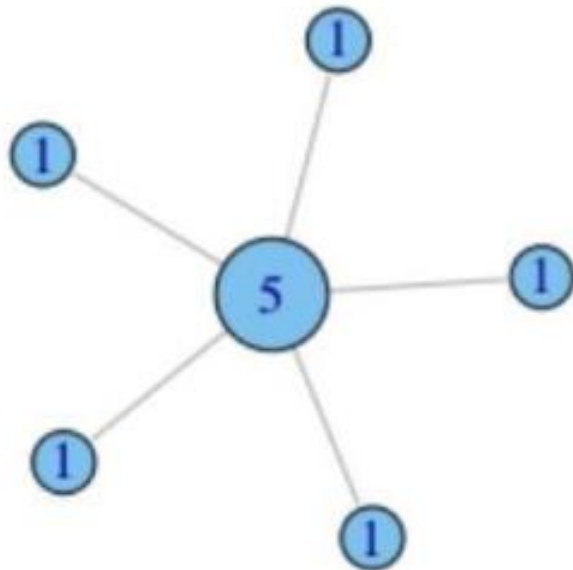


betweenness



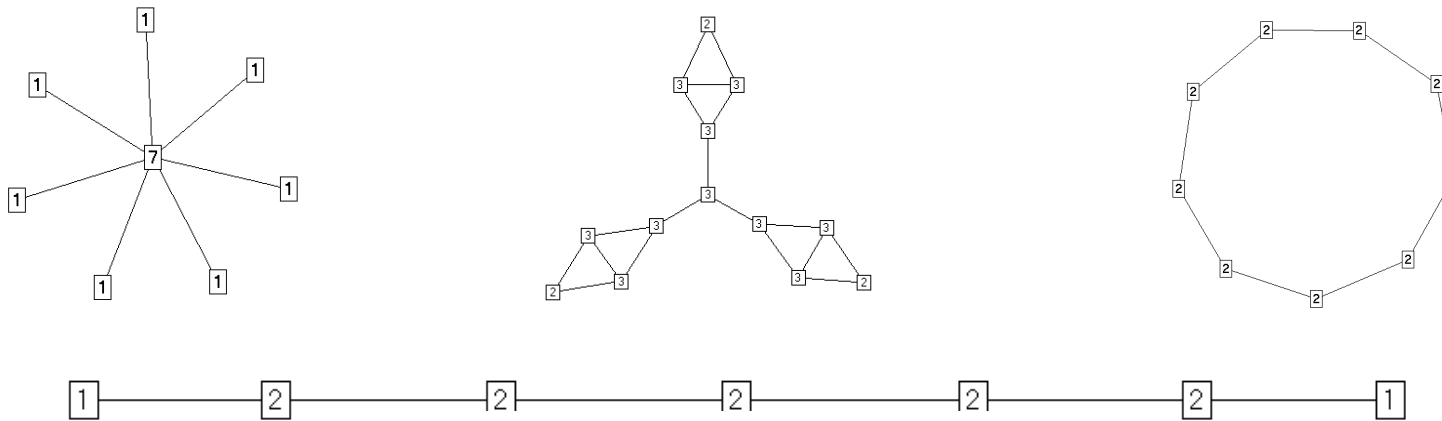
closeness

- People who have many friends is most important.
- When is the number of connections the best centrality measure?
 - People who will do favors for you
 - People you can talk to (influence set, information access, ...)
 - Influence of several actions



Visualization of Degree Centrality

- The most intuitive notion of centrality focuses on degree:
 - The actor with the most ties is the most important:

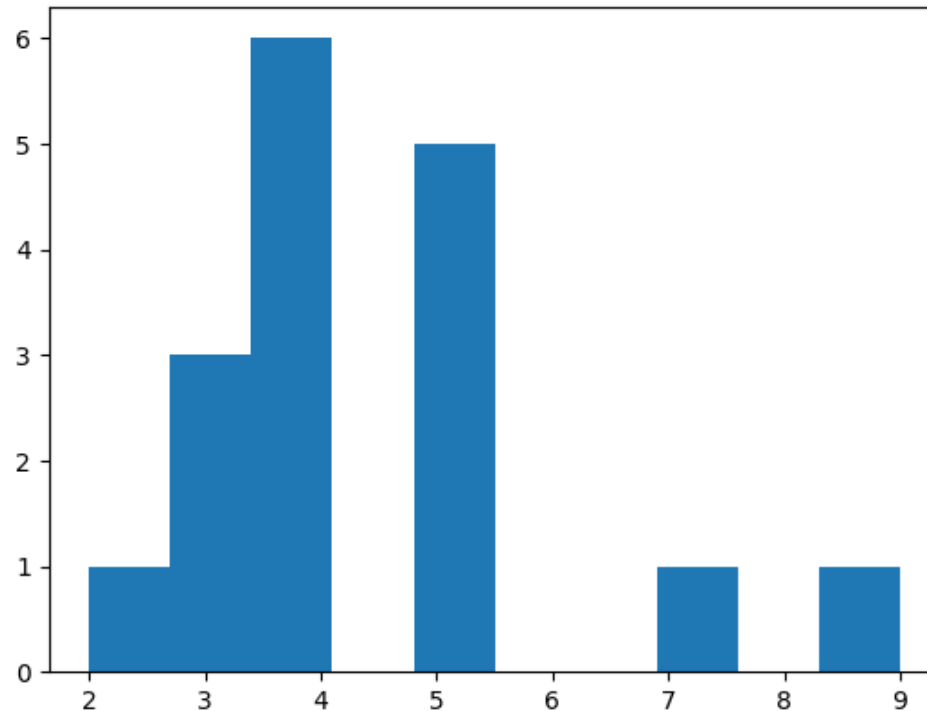


$$C_D(v_i) = d(v_i) = \sum_{j=1}^n A_{ij}$$

Where

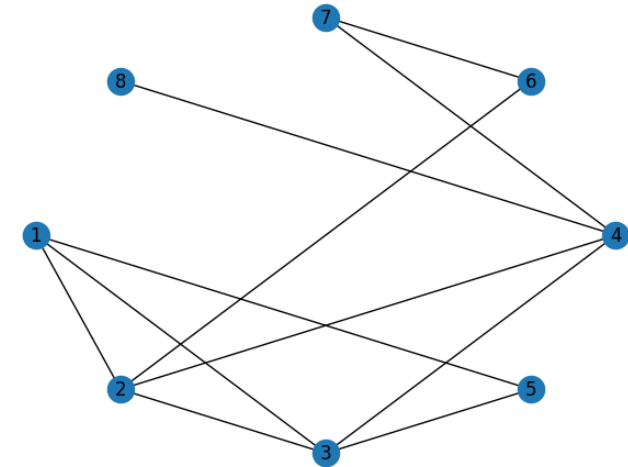
- $C_D(i)$: Degree centrality of node i (number of direct connections of node i)
- A_{ij} : Element of the adjacency matrix;
- $A_{ij} = 1$: if node i is connected to node j , otherwise 0
- n : Total number of nodes in the network

- In a simple random graph, degree will have a Poisson distribution, and the nodes with high degree are likely to be at the intuitive center.
- Deviations from a Poisson distribution suggest non-random processes, which is at the heart of current “**scale-free**” work on networks.



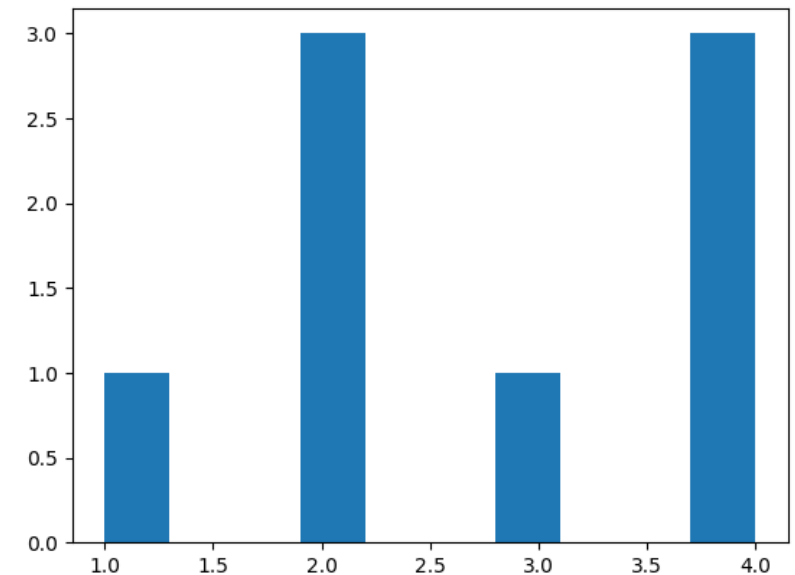
Constructing a Graph from an Edge List (*edge_list.txt*)

```
# Undirected graph
G = nx.read_edgelist('./data/edge_list.txt')
nx.draw(G, pos=nx.shell_layout(G), with_labels = True)
#nx.draw(G, pos=nx.spiral_layout(G), with_labels = True)
# Directed graph
# G = nx.read_edgelist('edge_list.txt', create_using=nx.DiGraph())
```



Histogram degree plot

```
degrees = [G.degree(n) for n in G.nodes()]
plt.hist(degrees)
```



➤ How much variation is there in the centrality scores for the entire network?

→ Measure the ***dispersion of node degree centralities***.

1 Variance of Degree Centrality

Definition

Let the degree sequence be

$$k_1, k_2, \dots, k_n$$

Mean degree:

$$\bar{k} = \frac{1}{n} \sum_{i=1}^n k_i$$

Degree variance:

$$Var(k) = \frac{1}{n} \sum_{i=1}^n (k_i - \bar{k})^2$$

Interpretation

- Low variance → degrees are similar
- High variance → some nodes have many more connections

➤ How much variation is there in the centrality scores for the entire network?

→ Measure the ***dispersion of node degree centralities***.

2 Freeman's Centralization (Normalized)

$$C_D(G) = \frac{\sum_{i=1}^n [C_D(v^*) - C_D(v_i)]}{(n-1)(n-2)},$$

where:

- v^* : the node with the **highest degree** in G

Interpretation:

- $C_D(G) = 0 \rightarrow$ All nodes have similar degree (Decentralized)
- $C_D(G) = 1 \rightarrow$ Star network (Fully centralized)

➤ How much variation is there in the centrality scores for the entire network?

→ Measure the ***dispersion of node degree centralities***.

3 Gini Coefficient

The Gini coefficient is calculated as:

$$G = \frac{\sum_{i=1}^N \sum_{j=1}^N |c_i - c_j|}{2N^2 \bar{c}}$$

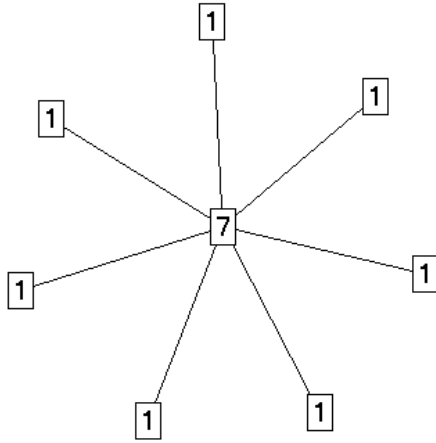
Where:

- N is the total number of nodes in the graph.
- c_i and c_j are the centrality scores (e.g., degree, betweenness, or eigenvector centrality) of nodes i and j .
- $\bar{c}$ is the average centrality score across all nodes.

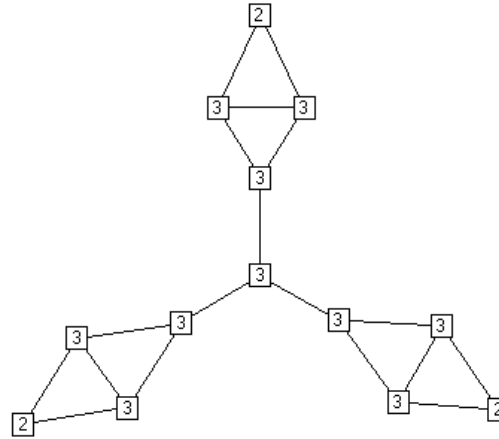
Interpretation:

- **0 (Perfect Equality):** Every node has the exact same centrality (e.g., a regular ring lattice or a complete graph).
- **1 (Perfect Inequality):** One node holds all the "importance" while others have none (approached by a large star graph)

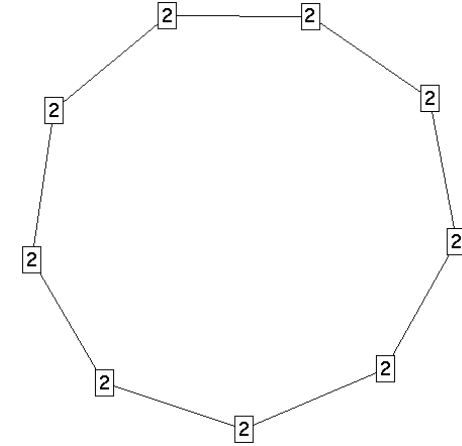
➤ Degree centralization scores



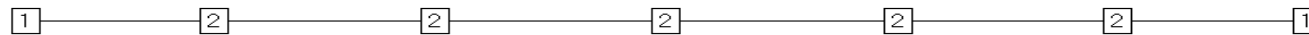
Freeman: 1.0
Variance: 3.9
Gini: 0.375



Freeman: 0.02
Variance: 0.17
Gini: 0.0641

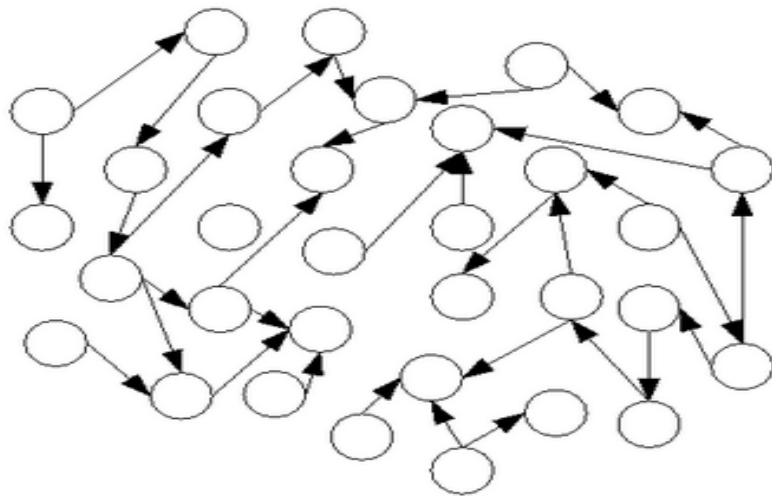


Freeman: 0.0
Variance: 0.0
Gini: 0.0



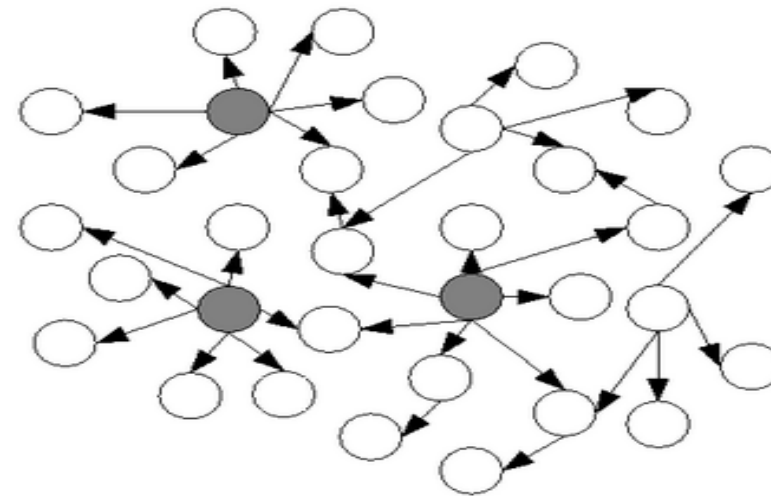
Freeman: 0.07
Variance: 0.20
Gini: 0.119

- Random network models introduce an edge between any pair of vertices with a probability p .
- The problem here is **NOT** randomness, but rather the distribution used (which, in this case, is uniform)



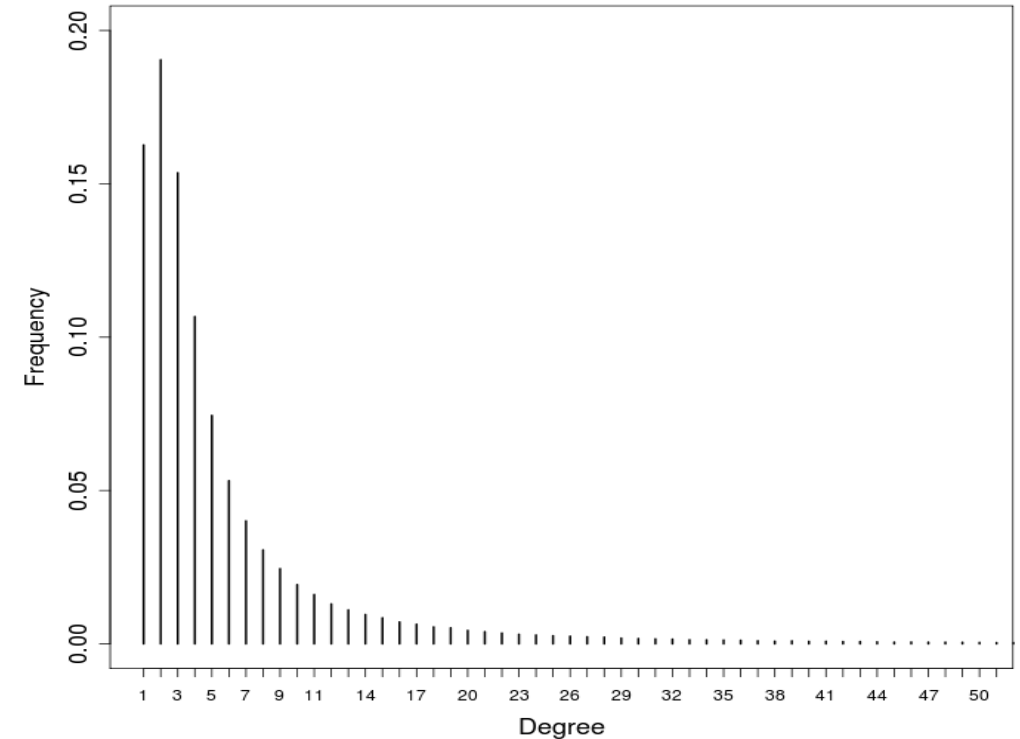
(a) Random network

- Real networks are not exactly like these:
 - Tend to have a relatively few nodes of high connectivity (the “Hub” nodes)
 - These networks are called “Scale-free” networks

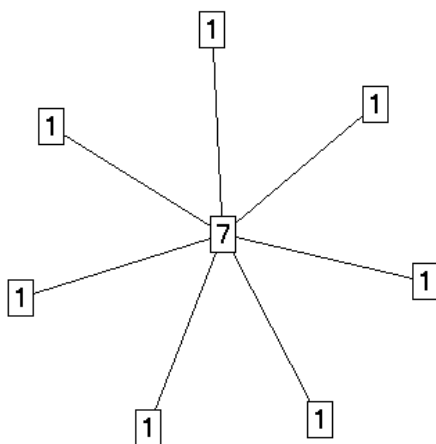


(b) Scale-free network

- In most real networks, the degree distribution is highly asymmetric:
 - Most of the nodes (the trivial many) have low degrees
 - A small but significant fraction of nodes (the vital few) have an extraordinarily high degree. A highly connected node, a node with remarkably high degree, is called hub showing a long tail.



➤ Degree centralization scores



Freeman: 1.0

Variance: 3.9

```
: import networkx as nx
import numpy as np
import matplotlib.pyplot as plt

# Instantiate the graph
G = nx.Graph()
# add node/edge pairs
G.add_edges_from([(1, 0), (1, 2), (1, 3), (1, 5), (1, 4), (1, 6), (1, 7)])
#Degree plot for undirected and weighted graph
degrees = [G.degree(n) for n in G.nodes()]

#degrees = [G.degree(n, weight='weight') for n in G.nodes()]
plt.hist(degrees)
mean = sum(degrees) / len(degrees)

v_start = max(degrees)
n = len(degrees)
var = sum((i - mean) ** 2 for i in degrees) / n
freeman = sum((v_start - i) for i in degrees) / ((n-1)*(n-2))
print(f'variance: {var}')
print(f'Freeman: {freeman}')
```

variance: 3.9375

Freeman: 1.0

- Using Freeman's general formula for centralization (which ranges from 0 to 1):

$$C_D(G) = \frac{\sum_{i=1}^n [d_{max} - d_i]}{(n-1)(n-2)},$$

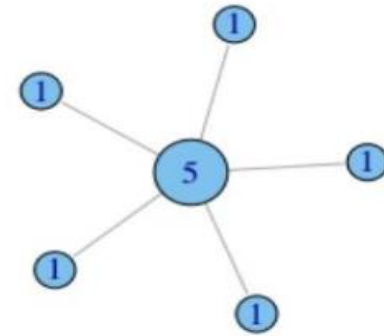
where:

- d_i : degree of node i
- d_{max} : maximum degree in the network
- n : total number of nodes



$$C_D = 0.167$$

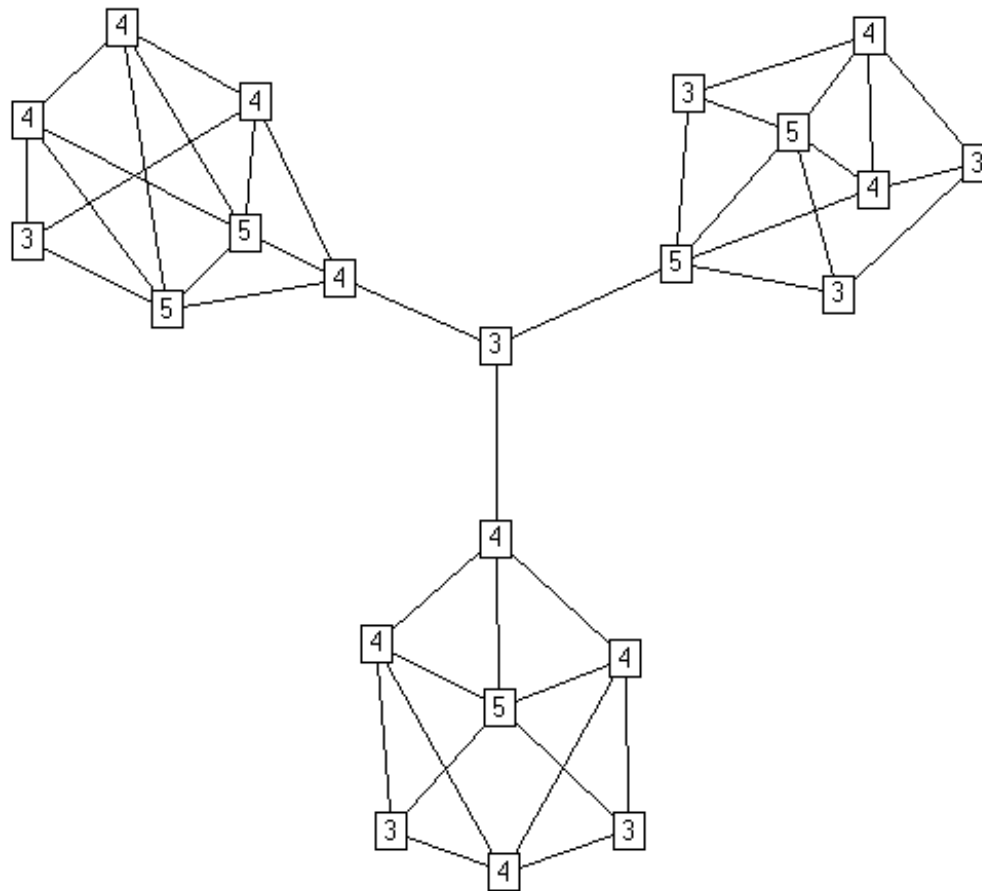
$$C_D(G) = \frac{(2-1) + (2-0) + \dots + (2-1)}{(5-1)(5-2)} = 0.167$$



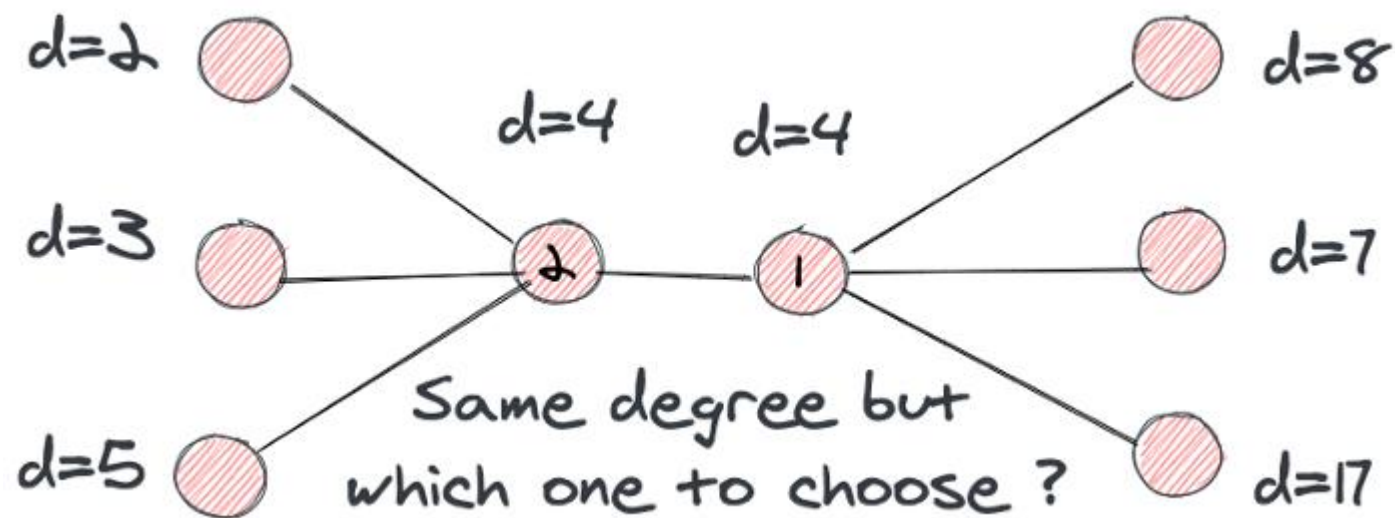
$$C_D = 1.0$$

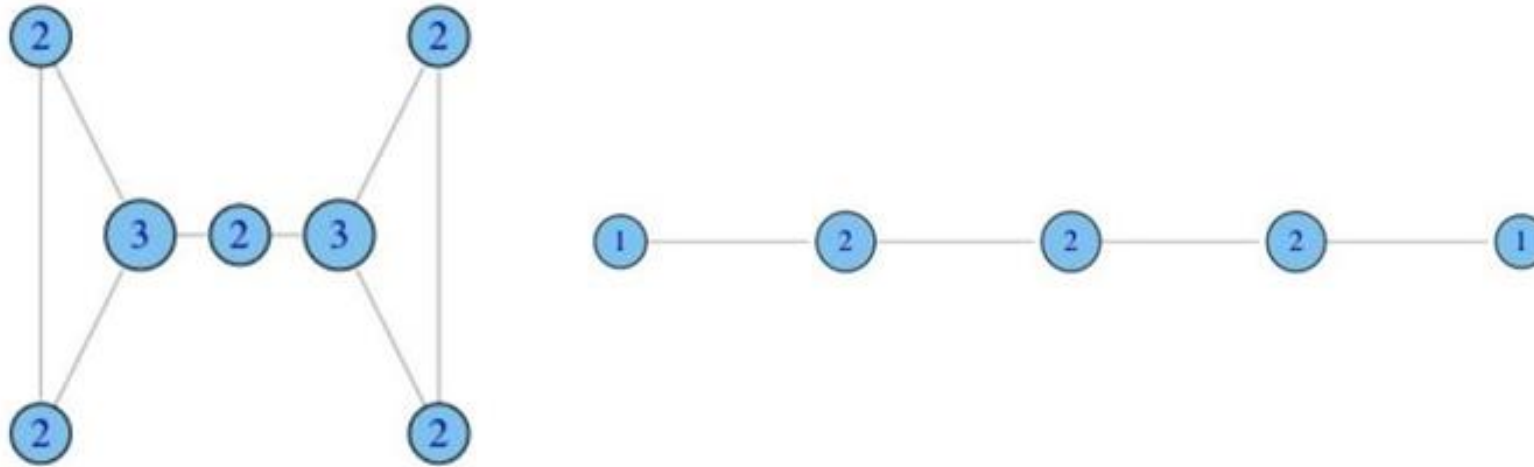
$$C_D(G) = \frac{(5-1) + \dots + (5-1)}{(6-1)(6-2)} = \frac{20}{20} = 1$$

- Degree centrality, however, can be deceiving, because it is a purely local measure



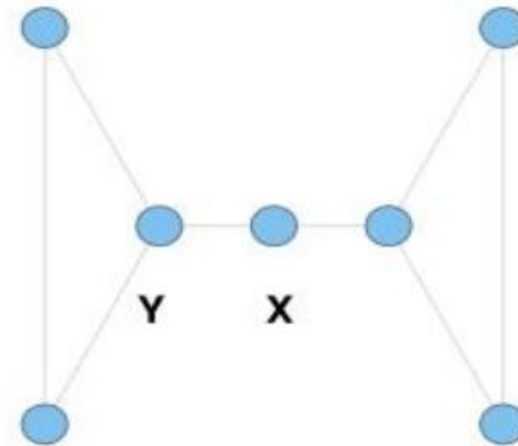
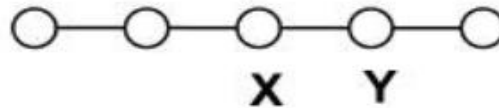
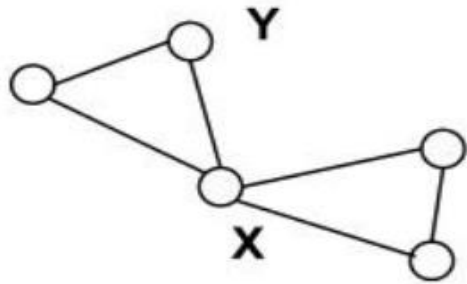
- Node degree is local, not global



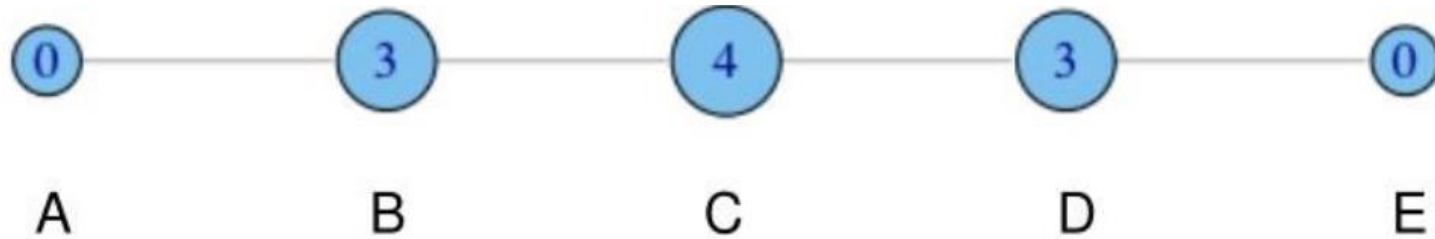


- Degree centrality measures only the number of direct links.
- It cannot capture nodes that act as bridges between groups.
- A node with fewer connections may still be more important if it connects different parts of the network.
- Therefore, degree centrality does not measure brokerage or control of information flow.

- What is **betweenness centrality**?
 - It measures **how often a node appears on the shortest paths between other nodes**
 - It identifies nodes that act as “bridges” or bottlenecks within network
- Who has higher betweenness, X or Y?



- Non-normalized version:
 - A lies between no two other nodes
 - B lies between A and 3 other nodes: C, D, E
 - C lies between 4 pairs: (A, D), (A, E), (B, D), (B, E)



- Betweenness centrality of node v_i :

$$B(v_i) = \sum_{v_j, v_k \in G} \frac{SPD_{v_j \rightarrow v_k}(v_i)}{SPD_{v_j \rightarrow v_k}}$$

The number of shortest paths between v_j and v_k that pass through the vertex v_i

The number of shortest paths from v_j to v_k

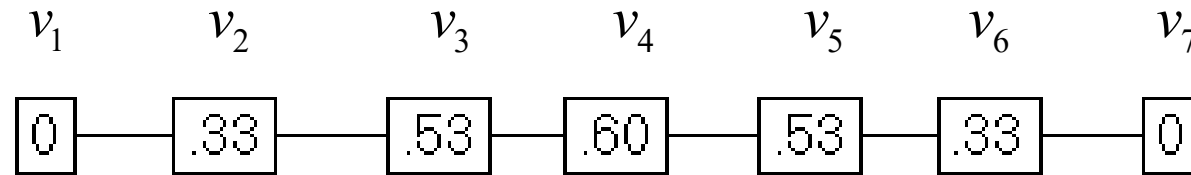
- Usually normalized by:

$$\bar{B}(v_i) = B(v_i) / [(n-1)(n-2) / 2]$$

No. of nodes in the graph

- Nodes with high betweenness centrality have influence in the network by virtue of their control over information passing between others.
 - They get to see the messages as they pass through
 - They could get paid for passing the message along
- Thus, they get a lot of power: their removal would disrupt communication

➤ Betweenness centrality of node v_2 :



$$SPD_{v_1 \rightarrow v_3}(v_2) = 1, SPD_{v_1 \rightarrow v_4}(v_2) = 1, SPD_{v_1 \rightarrow v_5}(v_2) = 1, SPD_{v_1 \rightarrow v_6}(v_2) = 1, SPD_{v_1 \rightarrow v_7}(v_2) = 1$$

$$SPD_{v_1 \rightarrow v_3} = 1, SPD_{v_1 \rightarrow v_4} = 1, SPD_{v_1 \rightarrow v_5} = 1, SPD_{v_1 \rightarrow v_6} = 1, SPD_{v_1 \rightarrow v_7} = 1$$

$$B(v_2) = \sum_{v_j, v_k \in G} \frac{SPD_{v_j \rightarrow v_k}(v_2)}{SPD_{v_j \rightarrow v_k}} = \frac{1}{1} + \frac{1}{1} + \frac{1}{1} + \frac{1}{1} + \frac{1}{1} = 5$$

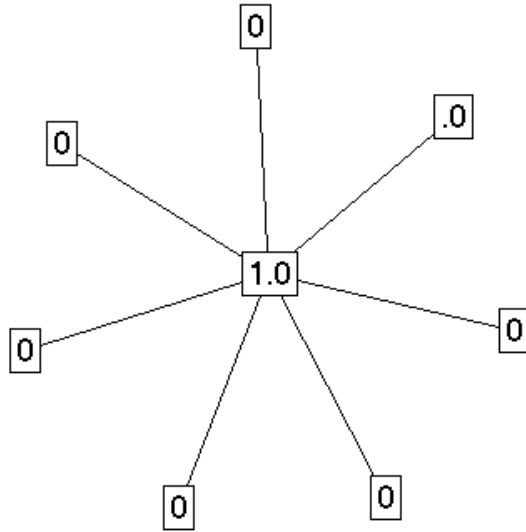
$$\bar{B}(v_2) = \frac{B(v_2)}{(n-1)(n-2)/2} = \frac{5}{(7-1)(7-2)/2} = 0.33$$

➤ Centralization:

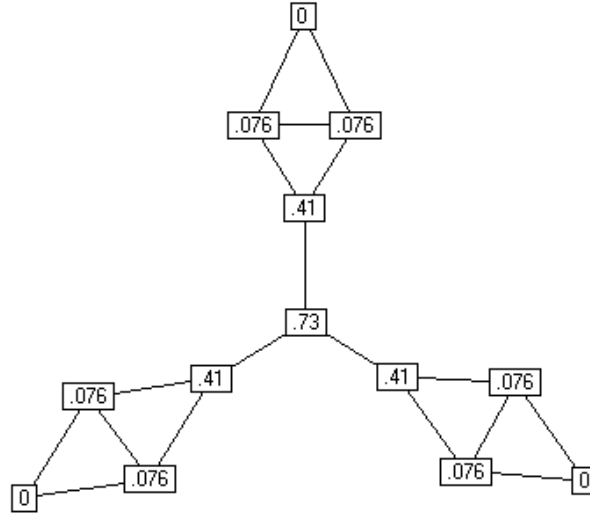
$$C_D(G) = \frac{\sum_{i=1}^n [B(v^*) - B(v_i)]}{(n-1)},$$

where:

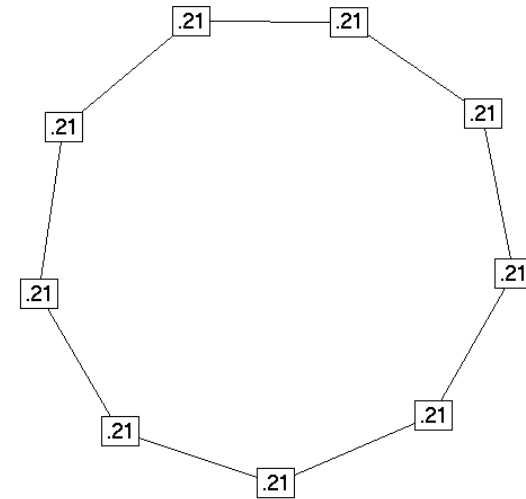
- v^* is the node with the highest betweenness in G
- B : Betweenness centrality



Centralization: 1.0



Centralization: 0.59



Centralization: 0



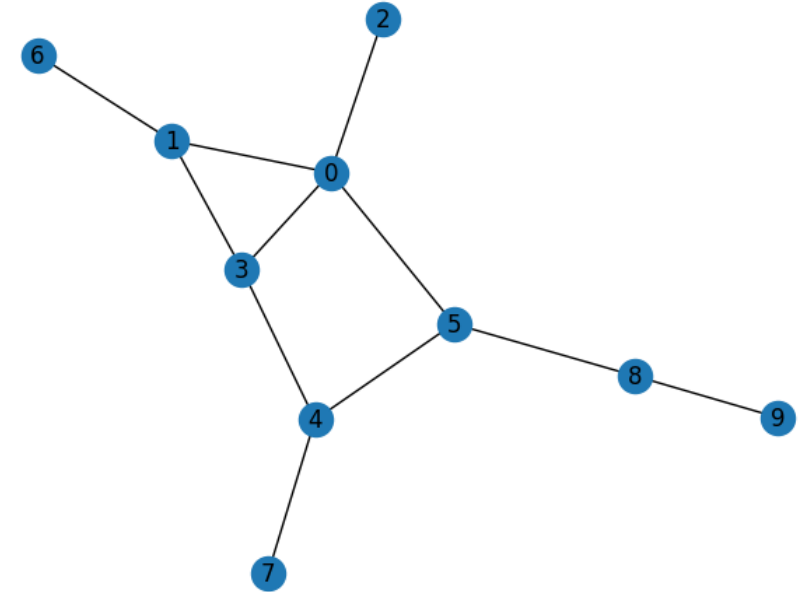
Centralization: 0.31

```
: import networkx as nx
import numpy as np
import matplotlib.pyplot as plt

# Instantiate the graph
G = nx.Graph()
# add node/edge pairs
G.add_edges_from([(0, 1),
                  (0, 2),
                  (0, 3),
                  (0, 5),
                  (1, 3),
                  (1, 6),
                  (3, 4),
                  (4, 5),
                  (4, 7),
                  (5, 8),
                  (8, 9)])
```

```
: #Sort for identifying most influential nodes using betweenness centrality
for node in sorted(betweenness centrality, key=betweenness centrality.get, reverse=True):
    print(node, betweenness centrality[node])
```

```
5 0.4444444444444444
0 0.4305555555555555
4 0.2638888888888889
1 0.2222222222222222
8 0.2222222222222222
3 0.16666666666666666
2 0.0
6 0.0
7 0.0
9 0.0
```



- Betweenness centrality is a measure of a node's influence over the flow of information in the network, often used to find nodes that serve as bridge between different network partitions
- Pros:
 - Entire network: The position of a node is related to the whole network
 - Flow perspective: It represents a different kind of importance compared to other centralities
- Cons:
 - Limit access: Access limited through only a few key persons might slow down the flow of information if some of resources or people become unavailable

- Problem:
 - What if: It is not so important to have many direct friends? Or be “between” others.
- But one still wants to be in the “middle” of things
 - Node v_i not too far from the center

- The closeness is defined so that if a node is close to every other node, then the value is larger than if the node is not close to everything else.

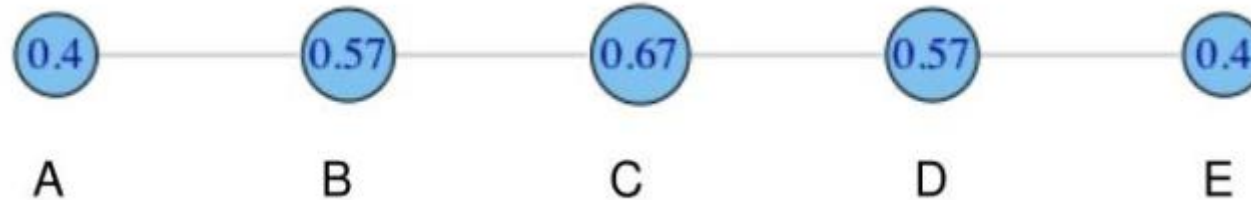
d: The number of nodes in the shortest path between 2 nodes

$$C(v_i) = \left[\sum_{j=1}^n |d(v_i, v_j)| \right]^{-1}$$

- Normalized Closeness Centrality:

$$\bar{C}(v_i) = \frac{n-1}{\sum_{j=1}^n |d(v_i, v_j)|}$$

- The Closeness centrality of node A = Sum of the number of nodes between A and other nodes:
 - $SPD(A, B) = 1$
 - $SPD(A, C) = 2$
 - $SPD(A, D) = 3$
 - $SPD(A, E) = 4$



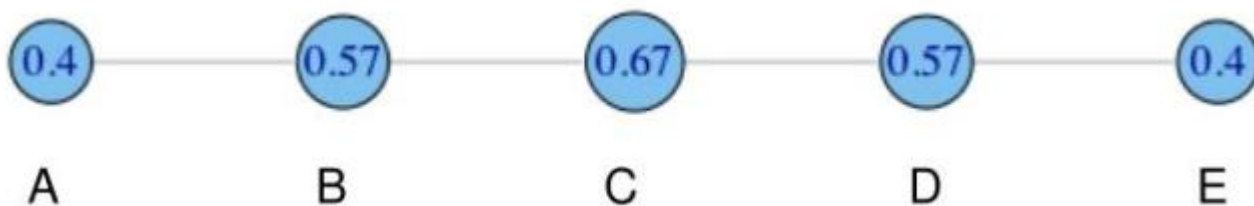
$$\bar{C}(A) = \frac{N - 1}{\sum_{j=1}^n |d(v_i, v_j)|} = \frac{4}{1 + 2 + 3 + 4} = 0.4$$

```
G = nx.Graph()
# add node/edge pairs
G.add_edges_from([(0, 1),
                  (1, 2),
                  (2, 3),
                  (3, 4)])

nx.draw(G, with_labels = True)

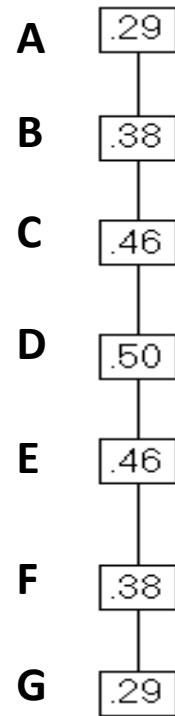
closeness centrality = nx.closeness centrality(G)
print(closeness centrality)
#Sort for identifying most influential nodes using closeness centrality
#for node in sorted(closeness centrality, key=closeness centrality.get, reverse=True):
# print(node, closeness centrality[node])
```

```
{0: 0.4, 1: 0.5714285714285714, 2: 0.6666666666666666, 3: 0.5714285714285714, 4: 0.4}
```



$$\bar{C}(A) = \frac{N - 1}{\sum_{j=1}^n |d(v_i, v_j)|} = \frac{4}{1 + 2 + 3 + 4} = 0.4$$

- Given a graph with 7 nodes: A, B, C, D, E, F, G
- Distance matrix denotes the shortest path distance between two nodes



Distance								Closeness	Normalized
A	B	C	D	E	F	G		.048	.286
B	0	1	2	3	4	5		.063	.375
C	1	0	1	2	3	4		.077	.462
D	2	1	0	1	2	3		.083	.500
E	3	2	1	0	1	2		.077	.462
F	4	3	2	1	0	1		.063	.375
G	5	4	3	2	1	0		.048	.286

- Comparing across these 3 centrality values
 - Generally, the 3 centrality types will be positively correlated
 - When they are not (low) correlated, it probably tells you something interesting about the network.

	Low Degree	Low Closeness	Low Betweenness
High Degree		Embedded in cluster that is far from the rest of the network	Ego's connections are redundant - communication bypasses him/her
High Closeness	Key player tied to important important/active alters		Probably multiple paths in the network, ego is near many people, but so are many others
High Betweenness	Ego's few ties are crucial for network flow	Very rare cell. Would mean that ego monopolizes the ties from a small number of people to many others.	

- An extension of degree centrality
 - Centrality increases with number of neighbors
- Not all neighbors are equal
 - Having connection to more central nodes increases importance
 - For example, a node with 300 relatively unpopular friends on Facebook would have lower eigenvector centrality than someone with 300 very popular friends
- Eigenvector centrality can help answer these questions:
 - Who or what holds wide-reaching influence in my network?
 - Who or what is important in my network on a macro scale?

$$Ax = \lambda x$$

Where:

- A = adjacency matrix of the network
- x = vector of eigenvector centrality scores
- λ = eigenvalue (specifically the **largest eigenvalue**)
- x_i = centrality score of node i

Equivalent Node-wise Expression

For each node i :

$$x_i = \frac{1}{\lambda} \sum_j A_{ij} x_j$$

Meaning:

- The centrality of node i equals the **sum of the centralities of its neighbors**, scaled by $1/\lambda$.

Intuition

- **Degree centrality**: counts number of neighbors.
- **Eigenvector centrality**: weights neighbors by **their importance**.

- Compute Eigenvector centrality:

$$x_i(t) = \sum_j A_{ij} x_j(t-1)$$

with the centrality at time $t=0$ being $x_i(0) = 1, \forall i$
 A denotes the adjacency matrix.

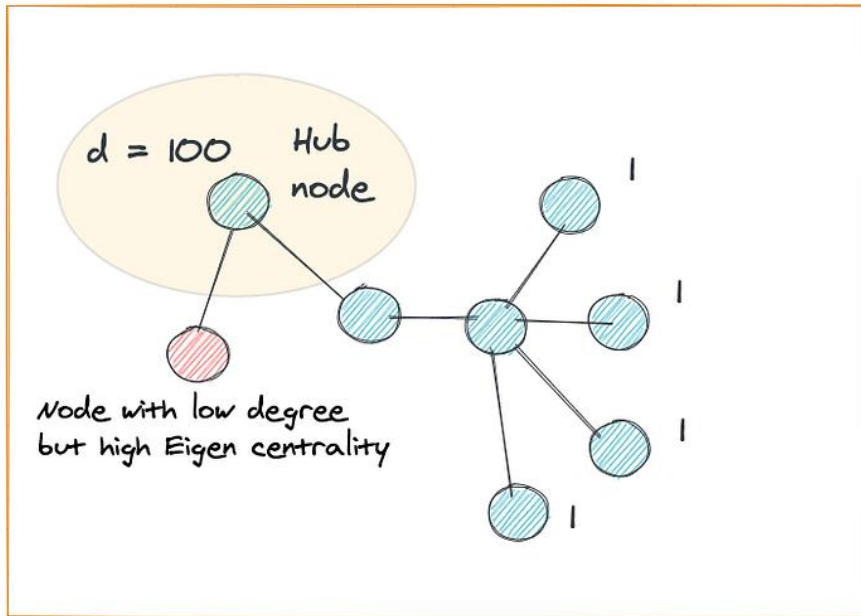
- Define the centrality x'_i of i recursively in terms of the centrality of its neighbours:

$$x'_i = \sum_{v_j \in N(v_i)} A_{ij} x_j \quad \text{with the initial node centrality } x_j = 1, \forall j$$

- That is equivalent to:

$$x_i(t) = \sum_{v_j \in N(v_i)} A_{ij} x_j(t-1) \quad \text{with the centrality at time } t=0 \text{ being } x_i(0) = 1, \forall i$$

The centrality of nodes x_i and x_j at time t and $(t-1)$, respectively.



Think about **joining Facebook**:

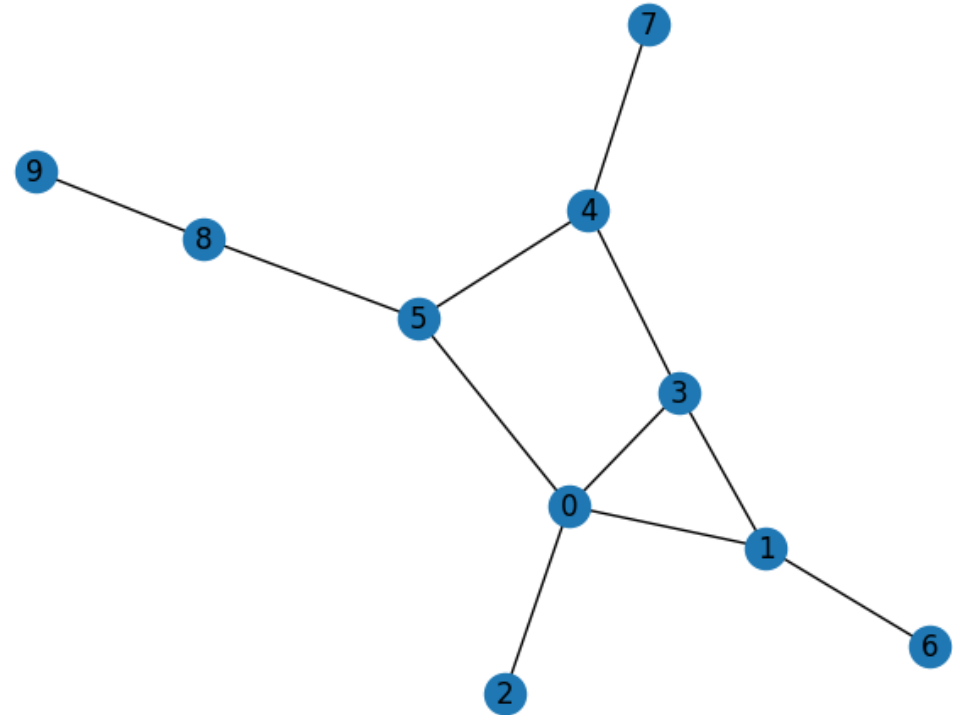
- You are **new to the network**.
 - You start **following Barack Obama** or a very popular person.
- This connection links you to an **important node**, but it **does not make you important in the network itself**.

```
import networkx as nx
import numpy as np
import matplotlib.pyplot as plt

# Create a graph
G = nx.Graph()
# add node/edge pairs
G.add_edges_from([(0, 1),
                  (0, 2),
                  (0, 3),
                  (0, 5),
                  (1, 3),
                  (1, 6),
                  (3, 4),
                  (4, 5),
                  (4, 7),
                  (5, 8),
                  (8, 9)])

# Calculate eigenvector centrality
eigenvector centrality = nx.eigenvector centrality(G)
# Sort for identifying most influential nodes using eigenvector centrality
for node in sorted(eigenvector centrality, key=eigenvector centrality.get, reverse=True):
    print(node, eigenvector centrality[node])
```

```
0 0.5163311132778224
3 0.4605589097129744
1 0.40781116737108153
5 0.3684701926326299
4 0.34608812874643025
2 0.18721071680314055
8 0.15382393197189426
6 0.14786314470285283
7 0.12548504820698178
9 0.05577442762250838
```



- Eigenvector centrality is an algorithm that measures the transitive influence of nodes.
- Relationships originating from high-scoring nodes contribute more to the score of a node than connections from low-scoring nodes.
- A high eigenvector score means that a node is connected to many nodes who themselves have high scores.
- Eigenvector centrality is the basis for several famous algorithms:
 - **Google PageRank** (modified eigenvector centrality)
 - **Katz centrality**
 - **HITS (authority/hub scores)**

- Katz centrality computes the centrality for a node based on the centrality of its neighbours. It is a generalization of the eigenvector centrality.
- The Katz centrality for node v_i is:

$$x_i = \alpha \sum_j A_{ij} x_j + \beta ,$$

where:

α is a constant called damping factor, and β is a bias constant,
 A is the adjacency matrix.

- When $\alpha = 1 / \lambda_{\max}$, $\beta = 0$, Katz centrality is the same as eigenvector centrality

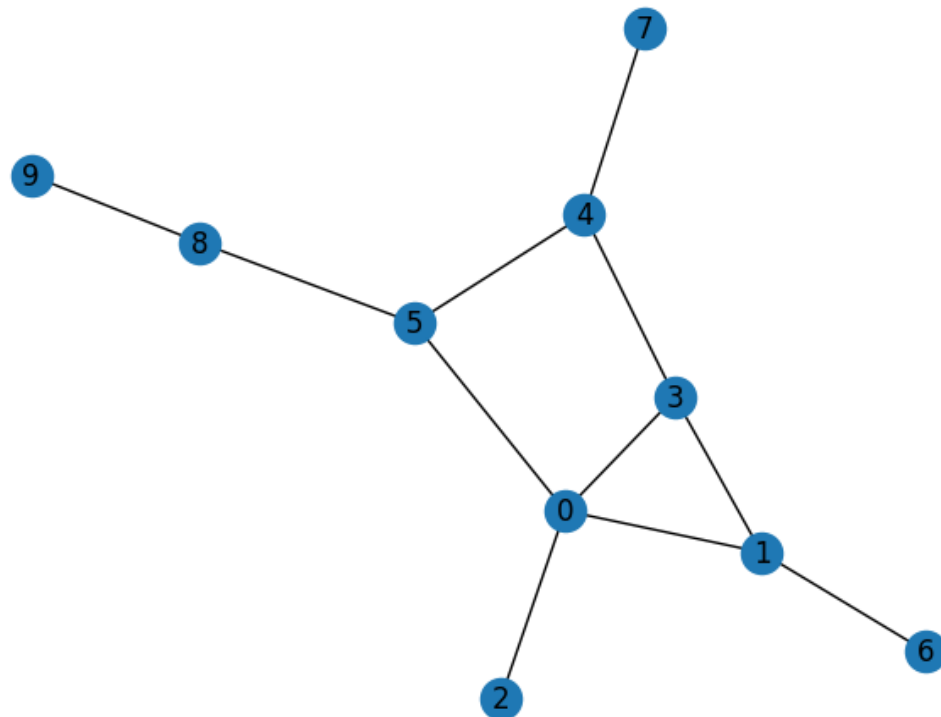
```
import networkx as nx
import numpy as np
import matplotlib.pyplot as plt

# Create a graph
G = nx.Graph()
# add node/edge pairs
G.add_edges_from([(0, 1),
                  (0, 2),
                  (0, 3),
                  (0, 5),
                  (1, 3),
                  (1, 6),
                  (3, 4),
                  (4, 5),
                  (4, 7),
                  (5, 8),
                  (8, 9)])
nx.draw(G, with_labels = True)
#Calculate kat centrality

kat centrality = nx.katz centrality(G)

#Sort for identifying most influential nodes using kat centrality
for node in sorted(kat centrality, key=kat centrality.get, reverse=True):
    print(node, kat centrality[node])
```

```
3 0.39837116164016045
2 0.3979957464558509
4 0.3912885477686667
1 0.3627994945952694
5 0.3266180854671848
6 0.32248848309558037
7 0.3218787410123867
8 0.28962990709484226
```



- Katz centrality is a network measure that considers all path between nodes, not just shortest ones. It penalizes connections with distant nodes
- Pros:
 - Considering all paths between nodes, not just the shortest ones, which can provide more comprehensive measure of influence within a network
 - It is like the eigenvector centrality, which are widely used and recognized measures
- Cons:
 - It can be computationally intensive for large networks as it involves calculating the total number of walks between all pairs of nodes
 - The attenuation factor, which penalizes connections with distant nodes, need to be chosen carefully. If not, it could skew the centrality measures

- Eigenvector centrality: i 's Rank score x_i is the sum of the Rank scores x_j of all example, j that point to i :

$$x_i = \sum_{(j,i) \in E} x_j$$

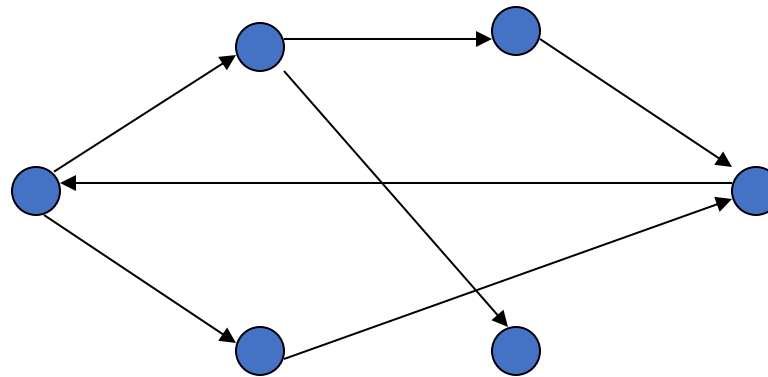
- Then Katz centrality adds **the teleportation** by adding a small weight edge to each node (using a weight of β):

$$x_i = \sum_{(j,i) \in E} x_j + \beta$$

- BUT, since a page j may point to many other pages, its prestige score should be shared among these pages.
(For example, one website can point to many sites)

$$x_i = \sum_{(j,i) \in E} \frac{x_j}{\text{out deg } x_j} + \beta$$

- Web pages are organized in a network.
 - Each webpage is represented as a node.
 - Each hyperlink is a directed edge
 - The entire web can be viewed as a directed graph.

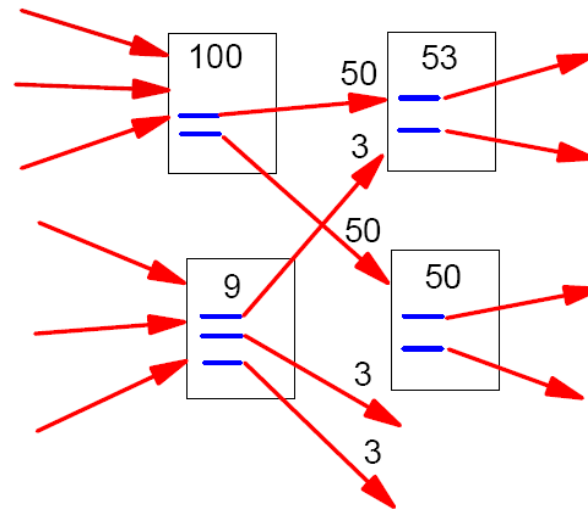


PageRank is a numeric value that represents how important a page is on the web.

- Webpage importance
 - One page links to another page = A vote for the other page A link from page A to page B is a vote on A to B.
 - If page A is more important itself, then the vote of A to B should carry more weight.
 - More votes = More important the page must be
- How can we model this importance?

➤ Importance Computation

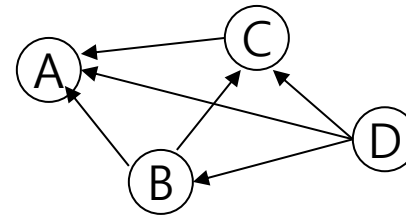
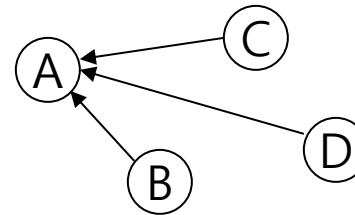
- The importance of a page is distributed to pages that it points to.
- The importance of a page is the aggregation of the importance shares of the pages that points to it.
 - If a page has 5 out links, the importance of the page is divided into 5 and each link receives one fifth share of the importance.



- Assume four web pages: A, B, C and D. Let each page would begin with an estimated PageRank of 0.25.

$$PR(A) = PR(B) + PR(C) + PR(D)$$

$$PR(A) = \frac{PR(B)}{2} + \frac{PR(C)}{1} + \frac{PR(D)}{3}$$



- $L(A)$ is defined as the number of links going out of page A. The PageRank of a page A is given as follows:

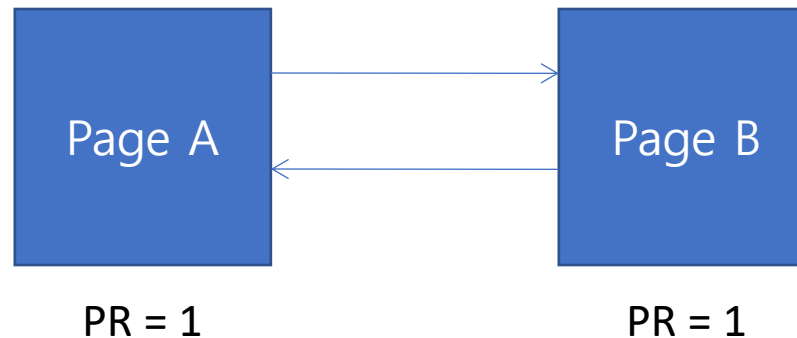
$$PR(A) = \frac{PR(B)}{L(B)} + \frac{PR(C)}{L(C)} + \frac{PR(D)}{L(D)}$$

- Assume page A has pages B, C, D ..., which point to it.
- The parameter d is a damping factor which can be set between 0 and 1.
- Usually set d to 0.85.
- The PageRank of a page A is given as follows:

$$PR(A) = 1 - d + d \left(\frac{PR(B)}{L(B)} + \frac{PR(C)}{L(C)} + \frac{PR(D)}{L(D)} + \dots \right)$$

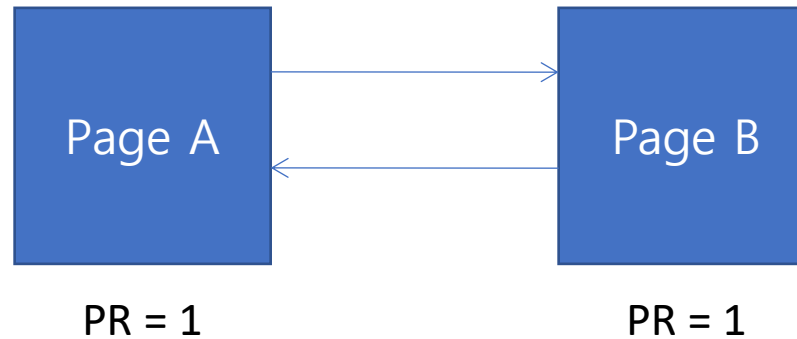
- Where:
 - d is a damping factor which can be set between 0 and 1 (usually set to 0.85)
 - $L(X)$ is the number of outbound links on page X

- We can't work out A's PageRank until we know B's PageRank, and we can't work out B's PageRank until we know A's PageRank.
- Iterations are necessary to calculate the most accurate values by using inaccurate values



$$\begin{aligned} PR(A) &= 0.15 + 0.85 * PR(B) \\ PR(B) &= 0.15 + 0.85 * PR(A) \end{aligned}$$

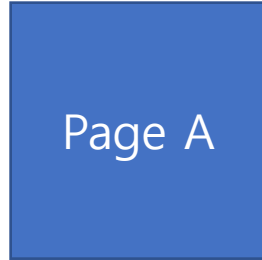
- A website has a maximum amount of PageRank that is distributed between its pages by internal links
- The maximum amount of PageRank in a site increases as the number of pages in the site increases
- By linking poorly, it is possible to fail to reach the site's maximum PageRank, but it is not possible to exceed it



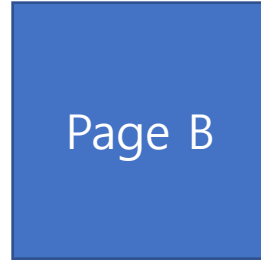
$$PR(A) = 0.15 + 0.85 * PR(B)$$

$$PR(B) = 0.15 + 0.85 * PR(A)$$

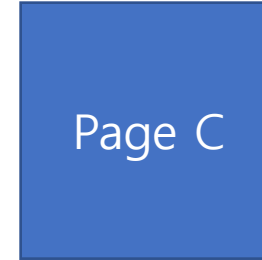
- Maximum PageRank is the amount of PageRank in the site.
- So, this site's maximum PageRank is 3.



PR = 1



PR = 1



PR = 1

$$PR(A) = 0.15 + 0.85 * (0) = 0.15$$

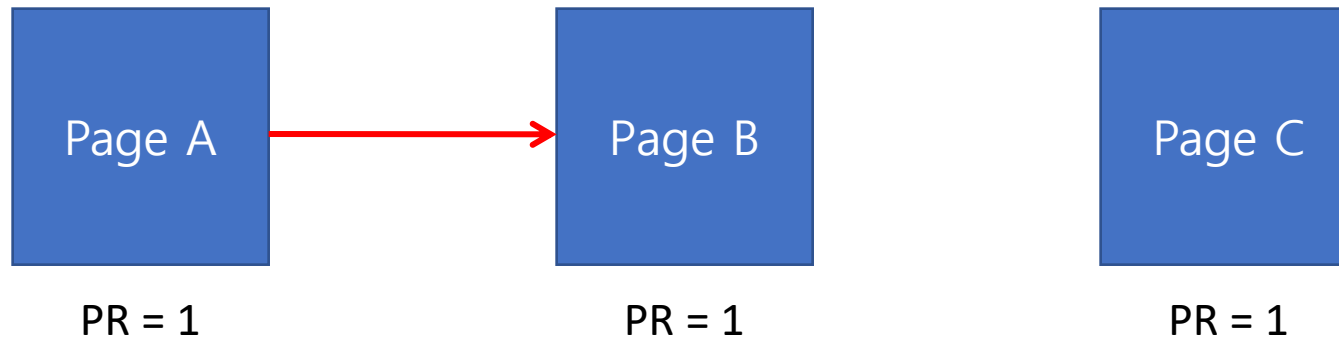
$$PR(B) = 0.15 + 0.85 * (0) = 0.15$$

$$PR(C) = 0.15 + 0.85 * (0) = 0.15$$

Total PageRank in this site = 0.45

Wasting most of its potential PageRank!

- Maximum PageRank is the amount of PageRank in the site.
- So, this site's maximum PageRank is 3.



$$PR(A) = 0.15 + 0.85 * (0) = 0.15$$

$$PR(B) = 0.15 + 0.85 * (PR(A)/1) = 0.15 + 0.85 * (1) = 1$$

$$PR(C) = 0.15 + 0.85 * (0) = 0.15$$

After 100 iterations....

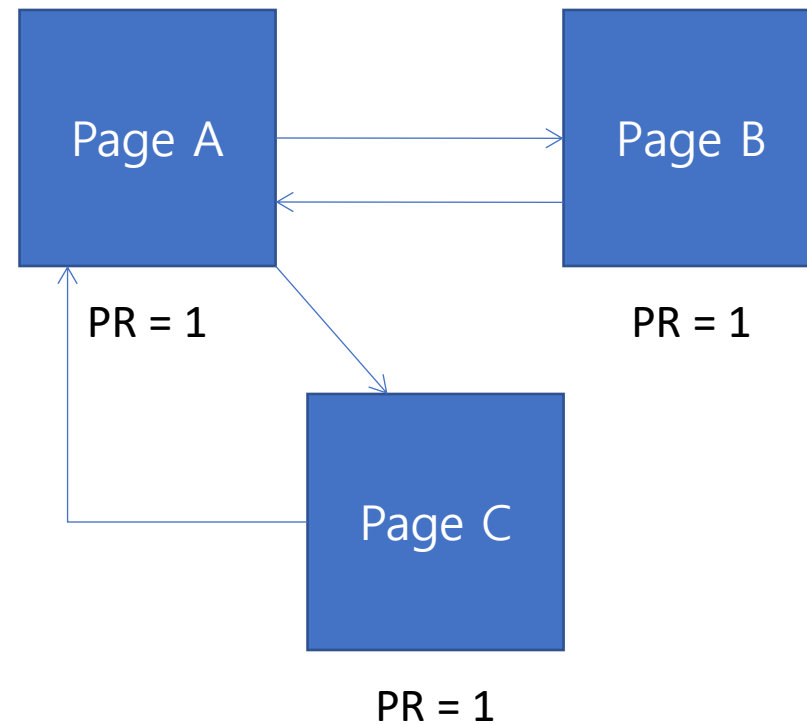
$$\begin{aligned} PR(A) &= 0.15 + 0.85 * (PR(B)/1 + PR(C) / 1) \\ &= 0.15 + 0.85 * (1 + 1) \\ &= 0.15 + 1.7 \\ &= 1.85 \end{aligned}$$

$$\begin{aligned} PR(B) &= 0.15 + 0.85 * (PR(A)/2) \\ &= 0.15 + 0.85 * (0.5) \\ &= 0.15 + 0.425 \\ &= 0.575 \end{aligned}$$

$$\begin{aligned} PR(C) &= 0.15 + 0.85 * (PR(A)/2) \\ &= 0.15 + 0.85 * (0.5) \\ &= 0.15 + 0.425 \\ &= 0.575 \end{aligned}$$

After 100 iterations...

Page A = 1.459459
Page B = 0.7702703
Page C = 0.7702703



```
import networkx as nx
import numpy as np
import matplotlib.pyplot as plt
```

```
# Create a graph
```

```
G = nx.Graph()
```

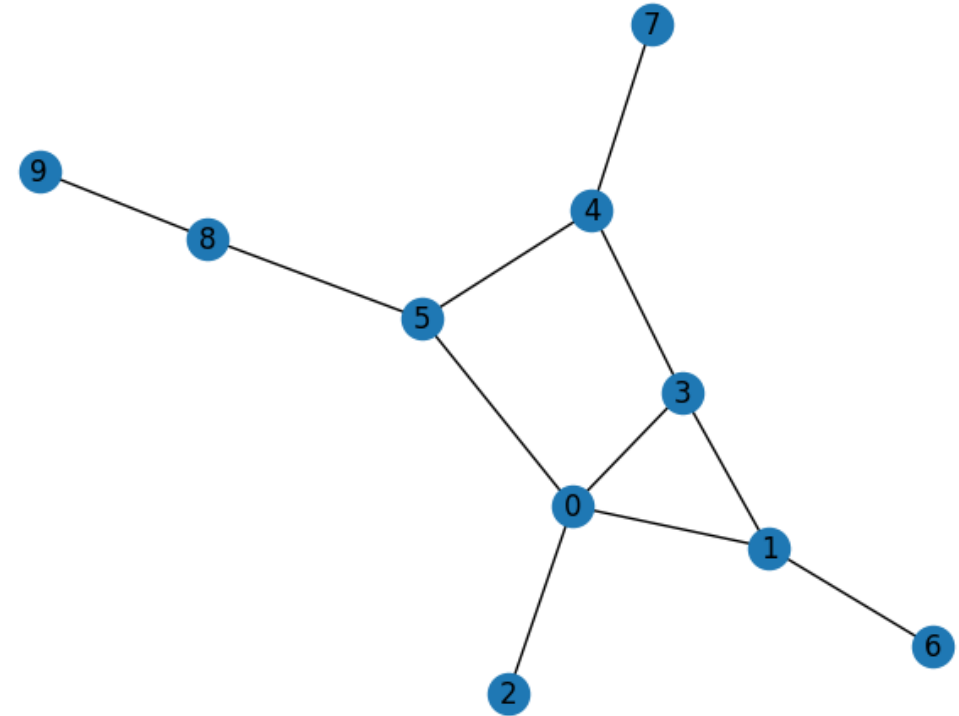
```
# add node/edge pairs
```

```
G.add_edges_from([(0, 1),
                  (0, 2),
                  (0, 3),
                  (0, 5),
                  (1, 3),
                  (1, 6),
                  (3, 4),
                  (4, 5),
                  (4, 7),
                  (5, 8),
                  (8, 9)])
```

```
#Calculate pagerank
```

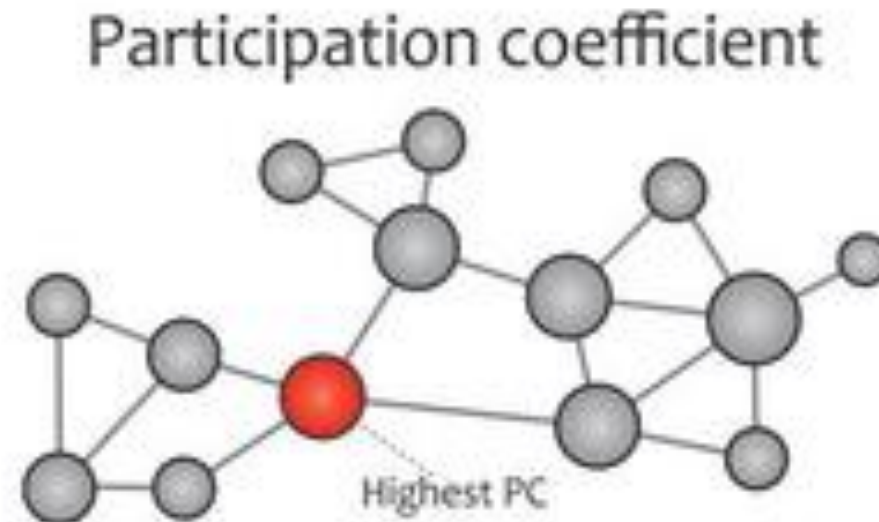
```
nx.pagerank(G, alpha=0.1)
```

```
{0: 0.10949439969135805,
1: 0.10541496772119344,
2: 0.09273734992283952,
3: 0.09978650527263376,
5: 0.10141741370884774,
6: 0.09351382767489713,
4: 0.10606037911522635,
7: 0.09353532767489713,
8: 0.1028950931069959,
9: 0.09514473611111113}
```



- Estimating Web Traffic
 - On analysing the statistics, it was found that there are some sites that have a very high usage, but low PageRank.
 - Ex: Links to pirated software
- PageRank as Backlink Predictor
 - The goal is to try to crawl the pages in as close to the optimal order as possible i.e., in the order of their rank.
 - PageRank is a better predictor than citation counting
- User Navigation: The PageRank Proxy
 - The user receives some information about the link before they click on it
 - This proxy can help users decide which links are more likely to be interesting

- In network science and graph theory, the participation coefficient is a metric used to measure how evenly a node's connections (edges) are distributed across different communities or modules within a network.
- It helps identify the specific role a node plays in a network - for instance, whether a node is strictly a "local player" communicating only within its own group, or a "connector" bridging multiple different groups



The participation coefficient P_i for a given node i is defined by the following formula:

$$P_i = 1 - \sum_{s=1}^{N_M} \left(\frac{k_{is}}{k_i} \right)^2$$

Where:

- N_M is the total number of modules (communities) in the network.
- k_{is} is the number of links (edges) node i shares with nodes in module s .
- k_i is the total degree (total number of connections) of node i .
- The sum $\sum$ is calculated over all modules in the network.

Interpret the Value

The participation coefficient always falls between **0** and **1** (strictly, approaching 1).

- $P_i = 0$ (**Low Participation**): All of node i 's connections are confined to its own module. The term $\frac{k_{is}}{k_i}$ for its own module is 1, and for all other modules is 0. Thus, $1 - 1^2 = 0$. This node is purely local.
- $P_i \rightarrow 1$ (**High Participation**): Node i 's connections are evenly distributed across all N_M modules in the network. It does not heavily favor any single community.

When combined with another metric called the **within-module degree z-score** (which measures how highly connected a node is within its own community), the participation coefficient helps classify nodes into distinct functional roles:

1. **Provincial Hubs**: High within-module connections, but a **low participation coefficient** (mostly connected to their own group).
2. **Connector Hubs**: High within-module connections and a **high participation coefficient** (important centers that also link different communities together).
3. **Kinless Hubs**: Extremely **high participation coefficient** (connections are so evenly spread across the whole network that they don't clearly belong to just one module).
4. **Satellite / Peripheral Nodes**: Low connections overall and a **low participation coefficient** (nodes at the edges of their specific community).

Centrality measures the importance or influence of nodes in a network.

Common Centrality Measures

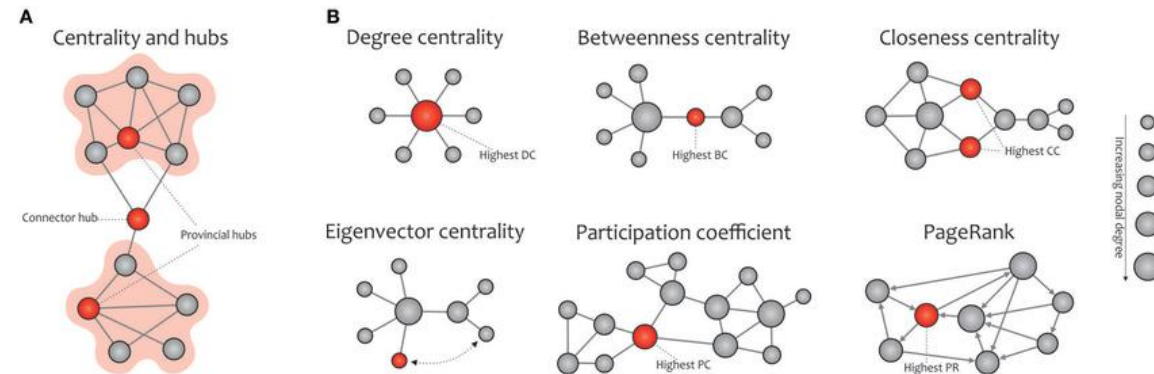
- Degree centrality
- Betweenness centrality
- Closeness centrality
- Eigenvector centrality

Distribution of Centrality

- Centrality values may be **unevenly distributed** across nodes.
- This can be analyzed using:
 - Centrality distributions
 - Network centralization

Centrality in Directed Networks

- In-degree centrality
- Out-degree centrality
- PageRank





네트워크 과학연구실
NETWORK SCIENCE LAB



가톨릭대학교
THE CATHOLIC UNIVERSITY OF KOREA

